

# DOSR: Decentralized Open-Source Review Project Proposal

William Convertino  
*wac20*

Jakub Romanczuk  
*jr614*

Gage Garcia  
*gv7*

Yuhan Wei  
*yw641*

## 1 Motivation

AI-assisted and AI-generated code has become an increasing burden for open-source software projects. Trusted maintainers may be unable to manually review a growing number of pull requests (PRs), and as agentic coding workflows become more capable, the number of automated PRs is growing. LLM-based PR review provides a natural way to automate this process [5, 8], but typically places the cost of inference on the repository maintainers.

This problem is especially challenging for *decentralized repositories*, where no single server or maintainer controls the canonical project state. Systems such as Radicle [9], Gitopia [2], and GOSH [3] distribute repository state and use mechanisms such as blockchain consensus or DAO voting to authorize updates, requiring many nodes to evaluate the same PR. As the number of PRs grows, this becomes unsustainable to maintain manually. Meanwhile, using LLM-based reviews makes each node responsible for running an independent evaluation, requiring the node owner to pay the associated API costs and leading to redundant evaluations across nodes.

## 2 Project Overview

To address these problems, we propose **DOSR (Distributed Open-Source Review)**, a framework for LLM-based PR approval in a decentralized Byzantine setting. Each repository defines a policy specifying approved LLM models, review format, and any additional approval requirements.

To submit a change, a contributor provides the PR, an approved LLM model API path, and their own API credentials. DOSR constructs the review request and uses TLSNotary [11] to produce a verifiable record binding the API request to the LLM response. If the model approves the change, the resulting certificate is distributed along with the PR. Other nodes can then verify the existing approval rather than independently recomputing the LLM evaluation.

This separates expensive *PR evaluation* from distributed *repository consensus*. The submitting contributor pays the

inference cost, while the decentralized protocol is responsible only for verifying the approval and maintaining agreement on repository state. This reduces redundant LLM computation, avoids burdening the non-contributor nodes, and provides an economic barrier against large numbers of low-effort submissions.

We will prototype DOSR and evaluate its end-to-end approval latency, certificate-generation and verification overhead, PR propagation latency as the number of nodes increases, and the cost savings relative to independently evaluating the same PR across multiple nodes.

## 3 Related Work

**AI PR Review.** AI-assisted code review is becoming increasingly popular, with systems like Qodo/PR-Agent and Mira using LLMs to analyze PRs and suggest changes or approvals [5, 8]. This field is still very new, and the quality of AI generated PR reviews is inconsistent [1, 10], but DOSR focuses on the system-level application of these processes in the hopes that they will improve with time.

**Verifiable LLM Calls.** TLSNotary provides cryptographic proof of data exchanged with a TLS server [11]. This makes it possible for a third-party to verify a particular prompt-response pair. OpenGradient provides a similar verification using trusted execution environments [7]. DOSR applies these techniques to solve the problem of AI PR reviews in decentralized repositories, allowing nodes to verify the acceptance of a PR request without making the API call themselves.

**Decentralized Repositories.** Several systems explore decentralized source-code management. Radicle [9] provides peer-to-peer Git replication and uses signed references to determine canonical state, while GOSH [3], Gitopia [2], BDA-SCV [4], and other works [6] use blockchain-based infrastructure. Rather than relying on consensus for approving PRs, DOSR computes the approval once and only uses consensus algorithms for verifying the approved PR and maintaining the final repository state.

## References

- [1] Fannar Steinn Aðalsteinsson, Björn Borgar Magnússon, Mislav Milicevic, Adam Nirving Davidsson, and Chih-Hong Cheng. Rethinking code review workflows with llm assistance: An empirical study. In *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 488–497, 2025.
- [2] Gitopia. Storage. <https://docs.gitopia.com/storage/index.html>. Gitopia documentation. Accessed: 2026-09-09.
- [3] GOSH. Organizations: GOSH DAO and Soft Majority Voting. <https://docs.gosh.sh/on-chain-architecture/organizations-gosh-dao-and-smv/>. GOSH documentation. Accessed: 2026-09-09.
- [4] Muhammad Hammad, Jawaid Iqbal, Ch Anwar ul Hassan, Saddam Hussain, Syed Sajid Ullah, Mueen Uddin, Urooj Ali Malik, Maha Abdelhaq, and Raed Alsaqour. Blockchain-based decentralized architecture for software version control. *Applied Sciences*, 13(5):3066, 2023.
- [5] miracodeai. Mira: Self-hosted AI code reviewer. <https://github.com/miracodeai/mira>. GitHub repository. Accessed: 2026-09-09.
- [6] Nishara Nizamuddin, Khaled Salah, Muhammad Ajmal Azad, Junaid Arshad, and M. H. Rehman. Decentralized document version control using ethereum blockchain and ipfs. *Computers & Electrical Engineering*, 76:183–197, 2019.
- [7] OpenGradient. TEE Gateway: TEE-secured inference node for third-party LLM inference requests. <https://github.com/OpenGradient/tee-gateway>. GitHub repository. Accessed: 2026-09-09.
- [8] Qodo. Qodo Merge (and open-source PR-Agent): Overview. <https://qodo-merge-docs.qodo.ai/index>. Accessed: 2026-09-09.
- [9] Radicle. Radicle Protocol Guide. <https://radicle.xyz/guides/protocol/>. Accessed: 2026-09-09.
- [10] Kexin Sun, Hongyu Kuang, Sebastian Baltes, Xin Zhou, He Zhang, Xiaoxing Ma, Guoping Rong, Dong Shao, and Christoph Treude. Does ai code review lead to code changes? a case study of github actions. *IEEE Transactions on Software Engineering*, 52(7):2111–2126, 2026.
- [11] TLSNotary. Notarization. <https://tlsnotary.org/docs/protocol/notarization/>. TLSNotary protocol documentation. Accessed: 2026-09-09.